

Wannier Projector Convention in the Adaptive Class-A Code

Repository note
(Dated: June 11, 2026)

This note records the convention used by `src/fgtn/classA.U1FGTN.py` and `src/fgtn/classA.U1FGTN.gpu.py` for the overcomplete-Wannier spinors and their associated rank-one projectors. The central point is that the code always forms a projector from the stored vector as $P = \chi_{\text{code}} \chi_{\text{code}}^\dagger$. However, the stored vector is the complex conjugate of a common ket convention for the real-space Wannier wavefunction. Thus, if one denotes the conventional Wannier ket by χ_W , the code projector is $(\chi_W \chi_W^\dagger)^*$.

CONTENTS

I. Band and measurement labels	1
II. Stored Wannier spinor	1
III. Rank-one projector	2
IV. Practical convention	2

I. BAND AND MEASUREMENT LABELS

The labels `p` and `m` in the code refer to positive- and negative-energy bands of the two-band Chern-insulator Bloch Hamiltonian, not to Bogoliubov particle and hole components. With

$$h(\mathbf{k}) = \hat{\mathbf{n}}(\mathbf{k}) \cdot \boldsymbol{\sigma}, \quad (1)$$

the code constructs

$$P_+(\mathbf{k}) = \frac{1}{2}(\mathbb{I} + h(\mathbf{k})), \quad P_-(\mathbf{k}) = \frac{1}{2}(\mathbb{I} - h(\mathbf{k})). \quad (2)$$

The arrays `WF_Ap` and `WF_Bp` are built from P_+ , while `WF_Am` and `WF_Bm` are built from P_- . In the postselected or feedback target state the positive-band modes are driven empty and the negative-band modes are driven occupied:

$$A_+, B_+ : \text{unoccupied}, \quad A_-, B_- : \text{occupied}. \quad (3)$$

The “hole” event in the measurement update is therefore the unoccupied outcome of the same single-particle mode; it is not represented by a separate complex-conjugated hole spinor.

II. STORED WANNIER SPINOR

Let τ be one of the local trial spinors, for example the σ_x eigenstates used by default. A common ket convention for a projected Wannier orbital centered at $\mathbf{R}$ is

$$\chi_W(\mathbf{r}; \mathbf{R}) = \mathcal{N}^{-1} \sum_{\mathbf{k}} e^{+i\mathbf{k} \cdot (\mathbf{r} - \mathbf{R})} P_{\pm}(\mathbf{k}) \tau. \quad (4)$$

This is a column spinor in the physical two-component internal space.

The code uses a different, but internally consistent, construction. In `construct_OW_projectors`, the momentum-space object is produced by

$$\psi_{\text{code},n}(\mathbf{k}) = \sum_m \tau_m^* [P_{\pm}(\mathbf{k})]_{mn}. \quad (5)$$

Since $P_{\pm}(\mathbf{k})$ is Hermitian, this is the complex conjugate of the usual projected ket component:

$$\psi_{\text{code},n}(\mathbf{k}) = \left[\sum_m [P_{\pm}(\mathbf{k})]_{nm} \tau_m \right]^* = (P_{\pm}(\mathbf{k})\tau)_n^*. \quad (6)$$

The code then multiplies by a center phase $e^{+i\mathbf{k}\cdot\mathbf{R}}$ and applies `fft2` over the momentum axes. With the standard NumPy convention, `fft2` carries the kernel $e^{-i\mathbf{k}\cdot\mathbf{r}}$, up to the global FFT normalization. Hence the stored spinor has the form

$$\chi_{\text{code}}(\mathbf{r}; \mathbf{R}) \propto \sum_{\mathbf{k}} e^{-i\mathbf{k}\cdot\mathbf{r}} e^{+i\mathbf{k}\cdot\mathbf{R}} (P_{\pm}(\mathbf{k})\tau)^* = \sum_{\mathbf{k}} e^{-i\mathbf{k}\cdot(\mathbf{r}-\mathbf{R})} (P_{\pm}(\mathbf{k})\tau)^*. \quad (7)$$

Comparing Eqs. (4) and (7), one obtains

$$\chi_{\text{code}}(\mathbf{r}; \mathbf{R}) = \chi_{\text{W}}(\mathbf{r}; \mathbf{R})^* \quad (8)$$

up to the harmless overall normalization convention, which is removed when the code normalizes each center. Spatial shell truncation and domain-wall-sector truncation are then applied directly to this stored spinor and followed by renormalization.

III. RANK-ONE PROJECTOR

Once the vector is stored, there is no further ambiguity in the code. For a stored vector

$$\chi_{\text{code}} \equiv \text{WF_}^*[:, \text{Rx}, \text{Ry}], \quad (9)$$

the CPU implementation forms

$$P_{\text{code}} = \chi_{\text{code}} \chi_{\text{code}}^{\dagger}, \quad (10)$$

implemented as `np.outer(chi, chi.conj())`. The GPU implementation uses the same convention,

$$P_{\text{code},ij} = \chi_{\text{code},i} \chi_{\text{code},j}^*. \quad (11)$$

If the symbol χ in a calculation is defined to mean the stored code vector, then the correct statement is simply

$$P = \chi \chi^{\dagger}. \quad (12)$$

If instead χ denotes the conventional Wannier ket χ_{W} of Eq. (4), then Eq. (8) gives

$$P_{\text{code}} = |\chi_{\text{W}}^*\rangle \langle \chi_{\text{W}}^*| = (|\chi_{\text{W}}\rangle \langle \chi_{\text{W}}|)^*. \quad (13)$$

In components,

$$(P_{\text{code}})_{ij} = \chi_{\text{W},i}^* \chi_{\text{W},j} = (\chi_{\text{W},i} \chi_{\text{W},j}^*)^* = (P_{\text{W}})_{ij}^*. \quad (14)$$

IV. PRACTICAL CONVENTION

For code-facing derivations, it is safest to define

$$\chi \equiv \text{WF_}^*[:, \text{Rx}, \text{Ry}] \quad (15)$$

and use $P = \chi \chi^{\dagger}$ everywhere. For paper-facing derivations that start from the usual Fourier ket in Eq. (4), the translation to the present code is

$$\chi_{\text{code}} = \chi_{\text{paper}}^*, \quad P_{\text{code}} = P_{\text{paper}}^*. \quad (16)$$

The measurement branch, occupied versus unoccupied, is a separate issue from this complex-conjugation convention.